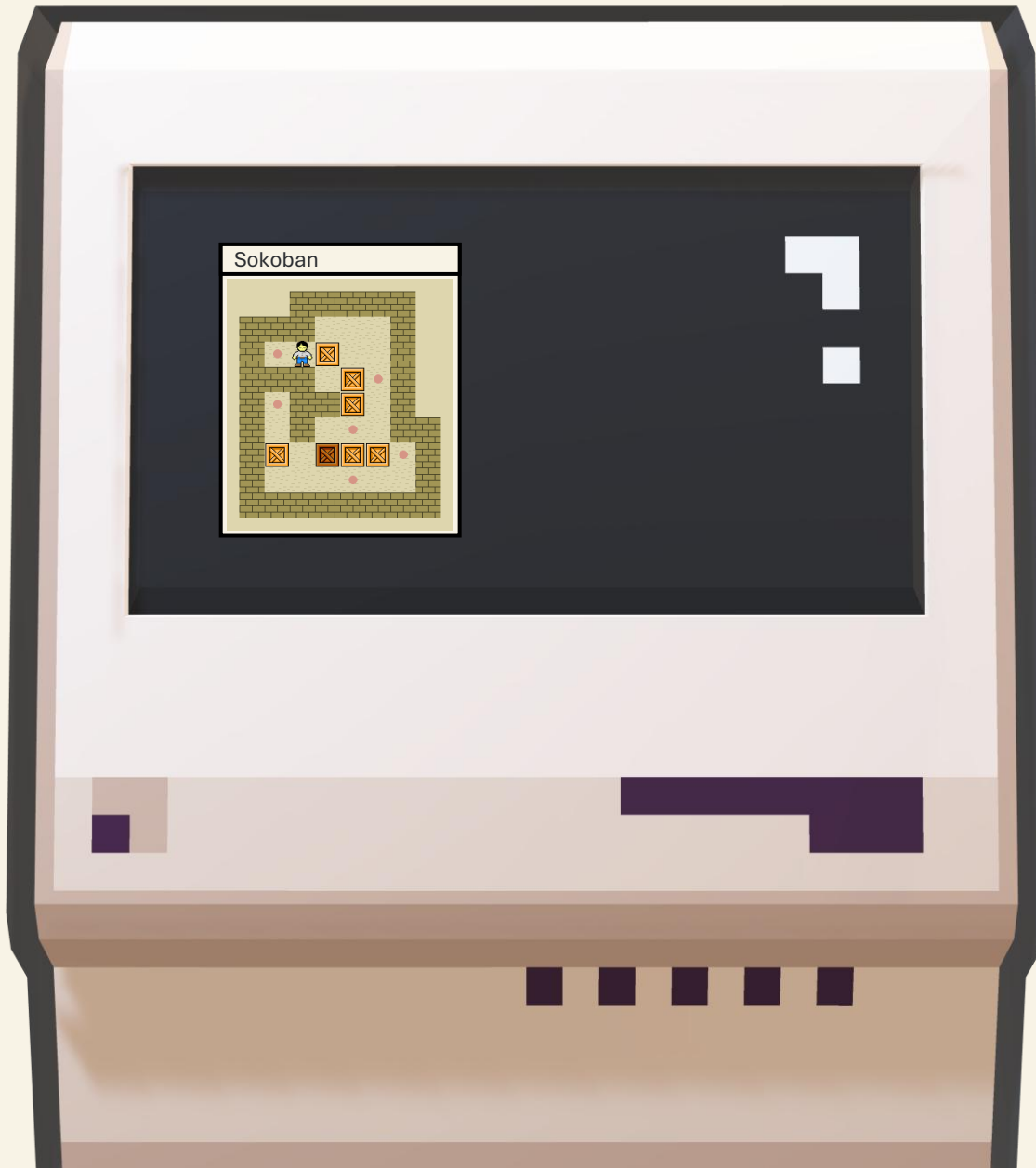


Team 2



Sokoban Search Agents

Team 2

“*UCS* vs *A**, a custom heuristic, and a competitive two-agent mode”

TEAM MEMBERS

524H0012 Trần Quang Duy Completion rate: 100%
524H0012@student.tdtu.edu.vn • Requirements 7 & 8

524H0017 Nguyễn Duy Khang Completion rate: 100%
524h0017@student.tdtu.edu.vn • Requirements 1 & 6

524H0013 Lý Phúc Hải Completion rate: 100%
524h0013@student.tdtu.edu.vn • Requirement 5

524H0015 Lý Tín Hòa Completion rate: 100%
524h0015@student.tdtu.edu.vn • Requirements 2 & 4

524H0011 Nguyễn Dương Tấn Đức Completion rate: 100%
524H0011@student.tdtu.edu.vn • Requirement 3 — Task 2

TASK ALLOCATION

Task	Description	Assigned To	Completion
Requirement 1	Formulate state-space search problem	Khang	100%
Requirement 2	Implement UCS & A*, propose heuristic	Hòa	100%
Requirement 3	Contrast time & space complexity experiments	Đức	100%
Requirement 4	Discuss heuristic properties, design experiment	Hòa	100%
Requirement 5	Single-agent OOP Pygame GUI	Hải	100%
Requirement 6	Reformulate 2-agent competitive problem	Khang	100%
Requirement 7	Design 2-agent competitive algorithm (<1000ms limit)	Duy	100%
Requirement 8	Reimplement competitive GUI	Duy	100%
Task 2	Presentation slides, demo video	Đức	100%

1. STATE-SPACE RULES:

1. State Space (S)

- **Static Elements:** Obstacles $W \subset \mathbb{Z}^2$ and target locations $D \subset \mathbb{Z}^2$ are static constants.
- **Dynamic State Definition:** Formally defined on a grid of $W_{\text{map}} \times H_{\text{map}}$ as $s = (P, B)$:
 - $P = (x_P, y_P) \in \mathbb{Z}^2$: Agent coordinate ($0 \leq x_P < W_{\text{map}}$, $0 \leq y_P < H_{\text{map}}$).
 - $B = \{(x_{B1}, y_{B1}), \dots, (x_{Bk}, y_{Bk})\} \subset \mathbb{Z}^2$: Immutable set of k box coordinates. Implemented as a frozenset for $O(1)$ lookup and invariance.
- **Theoretical Upper Bound:** $|S| \leq |Floor| \times (|Floor|-1)! / (k! \times (|Floor|-1-k)!)$.

1. STATE-SPACE RULES:

2. Initial State (S0)

- Parsed deterministically from input configuration file:
 $S0 = (P0, B0)$.
- $P0 = \{(x, y) \mid \text{Grid}(x, y) = 'A'\}$
- $B0 = \{(x, y) \mid \text{Grid}(x, y) \in \{'B', 'C'\}\}$
- $D = \{(x, y) \mid \text{Grid}(x, y) \in \{'D', 'C'\}\}$, $W = \{(x, y) \mid \text{Grid}(x, y) = '\%'\}$

1. STATE-SPACE RULES:

3. Action Space (As)

- **Actions:** Discrete orthogonal set $A = \{\text{North, South, East, West}\}$ with vectors: $v_{\text{North}} = (0, -1)$, $v_{\text{South}} = (0, 1)$, $v_{\text{East}} = (1, 0)$, $v_{\text{West}} = (-1, 0)$.
- **Admissibility Predicate:** $\text{Valid}(s, a) \Leftrightarrow \text{ValidMove}(s, v_a) \vee \text{ValidPush}(s, v_a)$.
- **ValidMove:** $\text{ValidMove}(s, v_a) \Leftrightarrow (P + v_a \notin W) \wedge (P + v_a \notin B)$.
- **ValidPush:** $\text{ValidPush}(s, v_a) \Leftrightarrow (P + v_a \in B) \wedge (P + 2v_a \notin W) \wedge (P + 2v_a \notin B)$.

1. STATE-SPACE RULES:

4. Transition Model (Result(s, a))

- **Deterministic mapping:** Result: $S \times A \rightarrow S$ yielding successor $s' = (P', B')$.
- **Move:** Result(s, a) = (P + v_a, B) if ValidMove(s, v_a).
- **Push:** Result(s, a) = (P + v_a, $(B \setminus \{P + v_a\}) \cup \{P + 2v_a\}$) if ValidPush(s, v_a).

5. Goal Test (GoalTests) & Path Cost (c)

- **Goal Test:** GoalTest(s) = True $\Leftrightarrow B \subseteq D$ (since $|B| = |D| = k$, $B \equiv D$).
- **Step Cost:** Uniform cost $c(s, a, s') = 1$, $\forall s \in S, a \in A(s)$.
- **Path Cost:** Cumulative cost $g(s) = \sum_{i=1}^m c(s_{(i-1)}, a_i, s_i) = m$.

2. HEURISTIC PROPOSALS:

1. Heuristic 1: Exact-Maze-Distance Bipartite Matching

- **Backward BFS:** Calculates exact shortest-path maze distance $w(b_i, d_j)$ from all walkable cells to targets D prior to search.
- **Bipartite graph:** Constructs a weighted graph between box positions B and targets D for state $s = (P, B)$.
- **Minimum-cost matching:** Applies the Hungarian algorithm to compute minimum-cost perfect matching $h(s) = \min_{\pi \in \Pi} \sum_{i=1}^k w(b_i, \pi(b_i))$.

2. HEURISTIC PROPOSALS:

2. Heuristic 2: Deadlock-Penalized Minimum Cost Assignment

- **Deadlock detection:** Detects irreversible static deadlocks, such as a box pushed into a non-target wall corner.
- **Penalty:** Returns $h(s) = \infty$ if a deadlock is detected.
- **Otherwise:** Returns the sum of exact maze distances from each box to its nearest unassigned target.

2. HEURISTIC PROPOSALS:

3. Theoretical Layout Trade-offs (A* vs UCS)

- **Open Maps:** A* with Bipartite Matching prunes the branching factor effectively.
- **Narrow Corridors:** High per-node overhead of complex heuristic calculation in constrained spaces may allow UCS to evaluate paths faster.

2. HEURISTIC PROPOSALS:

Theoretical analysis:

Uniform Cost Search (UCS)

- **Advantages:**
 - **Zero Heuristic Overhead:** Extremely low computational cost per node as no heuristic evaluation is required.
 - **Optimal Guarantee:** Always finds the optimal path by expanding nodes strictly in order of cumulative path cost $g(n)$.
- **Disadvantages:**
 - **Uninformed Radial Expansion:** Expands blindly in all directions without any guidance toward the target positions.
 - **Memory Exhaustion:** State space scales factorially based on floor layout, causing out-of-memory errors on large/open maps.

2. HEURISTIC PROPOSALS:

Theoretical analysis:

A Search (with Bipartite Matching)*

- **Advantages:**

- **Massive State Reduction:** Drastically prunes the search space by prioritizing states that move boxes closer to targets. Essential for complex maps.

- **Disadvantages:**

- **High Algorithmic Overhead:** Executing the Hungarian algorithm per state is computationally expensive, making A* slower than UCS on small, tightly constrained maps.

2. HEURISTIC PROPOSALS:

Theoretical analysis:

A Search (with Bipartite Matching)*

- **Advantages:**

- **Massive State Reduction:** Drastically prunes the search space by prioritizing states that move boxes closer to targets. Essential for complex maps.

3. EMPIRICAL COMPARISON REPORT (UCS VS A^*):

1. Objective

- **Objective:** Evaluate empirical performance of UCS versus A^* with Bipartite Matching across time and space metrics.

3. EMPIRICAL COMPARISON REPORT (UCS VS A*):

2. Test Environment

- **Environment:** Windows 11 26200, 13th Gen Intel(R) Core(TM) i7-13620H (2.40 GHz), 32 GB RAM, Python 3.14.8.
- **Metrics:** Wall-clock search time (time.perf_counter), peak allocation (tracemalloc), expanded nodes, and maximum frontier size.

3. EMPIRICAL COMPARISON REPORT (UCS VS A*):

3. Method

- **Method:** Tested on 9 maps ranging from map1_basic (7×5, 1 box) to map8_hard (12×7, 4 boxes) and map9_example (9×9, 7 boxes).

3. EMPIRICAL COMPARISON REPORT (UCS VS A*):

4. Results Summary

Map	Cost	Expanded Nodes UCS / A*	Search Time UCS / A*	Peak Memory UCS / A*
map1_basic	4	13 / 7	0.06 / 0.09 ms	0.008 / 0.009 MB
map3_two_boxes	8	441 / 74	1.38 / 1.04 ms	0.296 / 0.064 MB
map6_three_boxes	15	18,925 / 1,445	74.06 / 31.02 ms	10.34 / 1.03 MB
map8_hard	32	2,537,871 / 81,713	27.24 / 2.06 s	1046.4 / 43.16 MB
map9_example	34	2,736,902 / 50,170	32.06 / 3.08 s	1486.7 / 22.88 MB

3. EMPIRICAL COMPARISON REPORT (UCS VS A*):

5. Analysis and Conclusion

- **Performance Gap:** Expanded-node reduction grows from $\sim 2\times$ on the 1-box map to $54.55\times$ on map9_example.
- **Overhead:** Heuristic calculation accounts for 67%–85% of A* time, making UCS marginally faster on trivial 1-box maps.
- **Memory Bottleneck:** UCS exceeds 1.4 GB peak memory on complex maps, while A* consumes ~ 21.6 MB.
- **Conclusion:** A* dominates UCS in time and space complexity on all multi-box maps.

4. HEURISTIC REPORT (MATHEMATICAL PROOFS):

1. Selected Heuristic Definition

- **Exact-Maze-Distance Bipartite Matching:** $h(s) = \min_{\pi \in \Pi} \sum_{i=1}^k w(b_i, \pi(b_i))$.

2. Proof of Admissibility

- **Goal:** Prove $0 \leq h(s) \leq h^*(s)$.
- **Lower bound:** Each box b_i must reach target $d_{\{\pi^*(b_i)\}}$. Pushes required are bounded below by $w(b_i, d_{\{\pi^*(b_i)\}})$.
- **True minimal cost:** True minimal cost includes pushes plus agent repositioning steps: $h^*(s) \geq \sum_{i=1}^k w(b_i, d_{\{\pi^*(b_i)\}})$.
- **Conclusion:** Since $h(s)$ is the minimum sum over all permutations: $h(s) \leq \sum_{i=1}^k w(b_i, d_{\{\pi^*(b_i)\}}) \leq h^*(s)$.

4. HEURISTIC REPORT (MATHEMATICAL PROOFS):

3. Proof of Consistency

- **Goal:** Prove $h(s) \leq h(s') + c(s, a, s')$, where $c(s, a, s') = 1$.
- **Case 1 — move without pushing:** $B = B' \Rightarrow h(s) = h(s') \leq h(s') + 1$.
- **Case 2 — push b to b' :** $B' = (B \setminus \{b\}) \cup \{b'\}$. By shortest-path property, $w(b, \pi'(b')) \leq w(b', \pi'(b')) + 1$.
- **Conclusion:** A sub-matching gives $h_{\text{sub}}(s) \leq h(s') + 1$; since $h(s) \leq h_{\text{sub}}(s)$, $h(s) \leq h(s') + 1$.

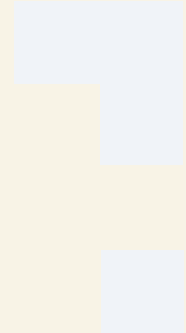
4. Search Algorithms — Pseudocode

A* SEARCH

```
AStarSearch(InitialState, Targets, Walls, H):  
  InitialH  $\leftarrow$  H(InitialState, Targets, Walls)  
  Frontier  $\leftarrow$  PQ((InitialState, 0, []), p=InitialH)  
  Explored  $\leftarrow$   $\emptyset$   
  WHILE Frontier  $\neq$   $\emptyset$ :  
    (State, Cost, Path)  $\leftarrow$  PopLowest()  
    IF State.Boxes = Targets: RETURN Path, Cost  
    IF State  $\notin$  Explored:  
      Explored  $\leftarrow$  Explored  $\cup$  {State}  
      FOR EACH (Next, Action, StepCost)  
        IN GetSuccessors(State, Walls):  
          IF Next  $\notin$  Explored:  
            NewCost  $\leftarrow$  Cost + StepCost  
            HCost  $\leftarrow$  H(Next, Targets, Walls)  
            IF HCost =  $\infty$ : CONTINUE // deadlock  
            Push((Next, NewCost, Path+[Action]),  
                priority=NewCost+HCost)  
  RETURN Failure
```

UNIFORM COST SEARCH

```
UniformCostSearch(InitialState, Targets, Walls):  
  Frontier  $\leftarrow$  PQ((InitialState, 0, []), p=0)  
  Explored  $\leftarrow \emptyset$   
  WHILE Frontier  $\neq \emptyset$ :  
    (State, Cost, Path)  $\leftarrow$  PopLowest()  
    IF State.Boxes = Targets: RETURN Path, Cost  
    IF State  $\notin$  Explored:  
      Explored  $\leftarrow$  Explored  $\cup$  {State}  
      FOR EACH (Next, Action, StepCost)  
        IN GetSuccessors(State, Walls):  
          IF Next  $\notin$  Explored:  
            NewCost  $\leftarrow$  Cost + StepCost  
            Push((Next, NewCost, Path+[Action]),  
                priority=NewCost)  
  RETURN Failure
```



5. SINGLE-AGENT GUI

1. Object-Oriented Programming (OOP) Model Explanation

The Pygame graphical interface utilizes an Object-Oriented Programming (OOP) architectural pattern that strictly separates Game Logic and State Management from UI Rendering. The system comprises three primary classes:

Agent Class

- **Encapsulates player character entities. Key features include:**
 - Attributes: Grid position (x, y), unique identifier (agent_id), display color, and a dynamic pos property.
 - Methods: move(dx, dy) independently updates the agent's spatial coordinates on the grid.

5. SINGLE-AGENT GUI

1. Object-Oriented Programming (OOP) Model Explanation

The Pygame graphical interface utilizes an Object-Oriented Programming (OOP) architectural pattern that strictly separates Game Logic and State Management from UI Rendering. The system comprises three primary classes:

GameLogic Class

Manages core game mechanics, state transitions, physics, and step history execution.

- **Attributes & Properties:** Maintains coordinates for walls, targets, boxes, agents list, grid dimensions, state-tracking variables (history, is_paused), and initial configurations.
- **Key Methods:** `load_map()` for native symbol parsing, `execute_move()` for validity evaluation and physics handling, `move_backward()` / `move_forward()` for step history navigation, `reset_logic()`, and `toggle_pause()`.

5. SINGLE-AGENT GUI

1. Object-Oriented Programming (OOP) Model Explanation

The Pygame graphical interface utilizes an Object-Oriented Programming (OOP) architectural pattern that strictly separates Game Logic and State Management from UI Rendering. The system comprises three primary classes:

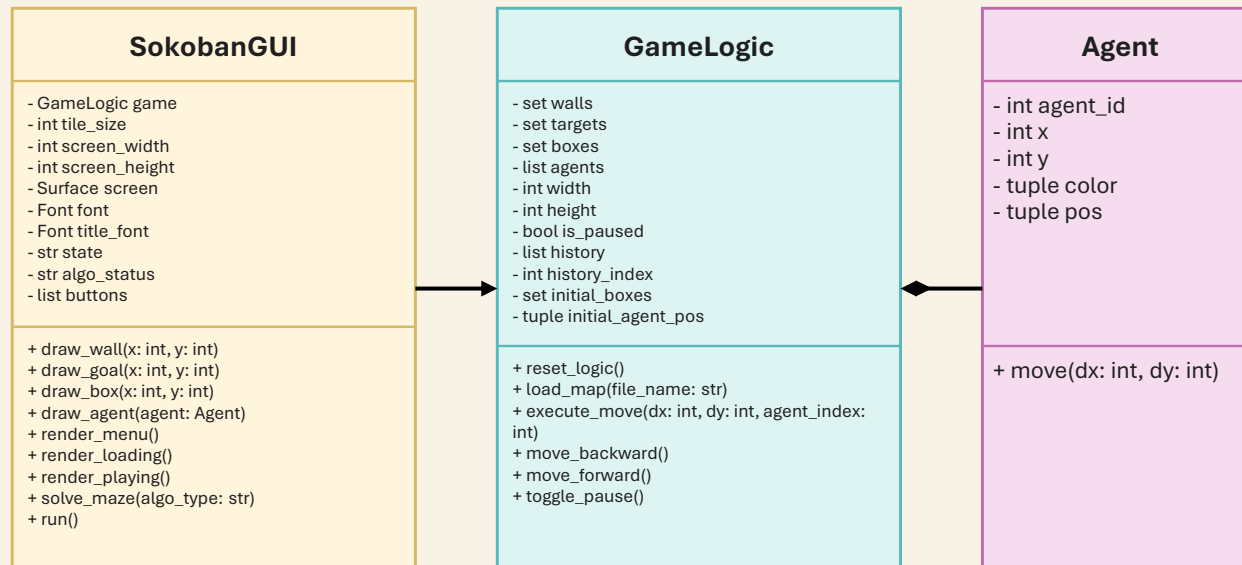
SokobanGUI Class

Handles all Pygame graphical output and user input event listening.

- **Attributes:** Holds a reference to the GameLogic instance, grid tile size, display surfaces, fonts, interactive buttons, and application state (MENU, LOADING, PLAYING).
- **Methods:** Modular rendering helpers (draw_wall, draw_goal, etc.), solve_maze(algo_type) to bridge UI and AI, state renderers, and run() for the main event loop.

5. SINGLE-AGENT GUI

Class Diagram



5. SINGLE-AGENT GUI

2. Advantages of the OOP Structure

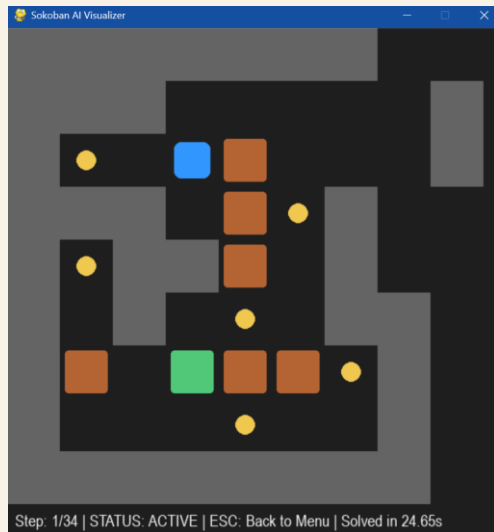
- **Decoupling UI & Logic:** GameLogic runs independently, allowing `solve_maze()` to perform calculations headlessly, preventing UI freezing during intensive A^* searches.
- **State Machine UX:** Provides a native, professional user experience inside Pygame without relying on terminal prompts.
- **Easier State Management:** Snapshot tuples in the history list allow $O(1)$ constant time complexity for stepping forward and backward.
- **Interactive Playback Control:** Fully meets requirements by letting users freeze state inspection via SPACE and navigate steps via LEFT/RIGHT arrows.

5. SINGLE-AGENT GUI

3. Disadvantages & Limitations

- **Memory Overhead:** Storing full position snapshots for every step in history increases memory usage, particularly for solutions exceeding 1,000 steps.
- **Direct Class Coupling:** SokobanGUI directly references GameLogic instead of using abstract interfaces, making it slightly more brittle to logic signature changes.

5. SINGLE-AGENT GUI



Step 1 / 34



Completed: 34 / 34

6. MULTI-AGENT RULES:

1. Competitive Multi-Agent State-Space (Scomp)

- **Game Model:** Simultaneous-move 2-player game: $G = \langle \text{Agents}, \text{Scomp}, A1 \times A2, T, n, U \rangle$.
- **6-Tuple State:** $\text{Scomp} = (P1, P2, B_{\text{free}}, B1, B2, t)$.
- **Positions:** $P1, P2 \in \mathbb{Z}^2$.
- **Owned boxes:** $B1, B2 \subset D$ are target-secured boxes; $B1 \cap B2 = \emptyset$.
- **Neutral boxes & time:** B_{free} contains boxes not on a target; $t \in \mathbb{N}$ and $0 \leq t \leq n$.

6. MULTI-AGENT RULES:

2. Simultaneous Execution & Collision Resolution Model (T)

- **Cell Congestion:** Both enter same cell ($P1 + v1 = P2 + v2$) $\Rightarrow$ both stay stationary.
- **Direct Swap Conflict:** Direct position swap ($P1 + v1 = P2 \wedge P2 + v2 = P1$) $\Rightarrow$ both stay stationary.
- **Static Body Obstacle:** Move into stationary opponent's cell $\Rightarrow$ cancelled.
- **Simultaneous Push Conflict:** Both push the same box concurrently $\Rightarrow$ box and agents stay stationary.

6. MULTI-AGENT RULES:

3. Box Stealing, Eviction & Ownership Transfer Rules

- **Scoring Placement:** Push neutral box $b \in B_{\text{free}}$ to target $d \in D \Rightarrow B_i \leftarrow B_i \cup \{d\}$ (+1 score).
- **Box Eviction / Stealing:** Agent j pushes box $b \in B_i$ off target d to non-target $p_{\text{new}} \notin D$.
- **Owner impact:** Owner i loses box: $B_i \leftarrow B_i \setminus \{d\}$ (score decrements by -1).
- **Neutral reset:** Box reverts to neutral: $B_{\text{free}} \leftarrow B_{\text{free}} \cup \{p_{\text{new}}\}$.
- **Re-scoring:** Any agent can push an evicted neutral box back to a target to claim ownership.

6. MULTI-AGENT RULES:

4. Step Limit (n) & Terminal Utility (U)

- **Terminal State:** Terminates strictly when cycle limit is reached: $t = n$.
- **Payoff Evaluation:** $\text{Score1} = |B1|$ and $\text{Score2} = |B2|$.
- **Utility Functions:** $U1 = \text{Score1} - \text{Score2}$; $U2 = \text{Score2} - \text{Score1}$.
- **Outcome:** The higher score wins; equal scores produce a draw.

7. COMPETITIVE SEARCH AGENTS:

1. Search Strategy

- **Algorithms:** A* and Greedy Best-First Search operate on an independent discrete state space.
- **Exact-maze-distance matching:** Static BFS distances route around walls, then permutations select the minimum box-to-target assignment.
- **Timeout failsafe:** The loop stops near 0.95 s and returns the most promising action before the 1,000 ms limit.

Advantages: Always returns an action; the pre-computed heuristic is more accurate than blind Euclidean distance.

Limitation: The $O(|B|!)$ matching step scales poorly and can force reliance on a sub-optimal fallback.

7. COMPETITIVE SEARCH AGENTS:

2. Search Procedure — Pseudocode

```
function Search(initial_state, algorithm):
    frontier.push(initial_state)
    best_fallback_action = "Wait"
    while frontier not empty AND elapsed < 0.95s:
        current = frontier.pop()
        if agent_scored_or_stole_box(current):
            return action_that_led_to(current)
        for successor in generate_legal_moves(current):
            h = Bipartite_Matching(successor.bboxes, targets)
            frontier.push(successor, g_cost + h)
            best_fallback_action = get_best_action()
    return best_fallback_action
```

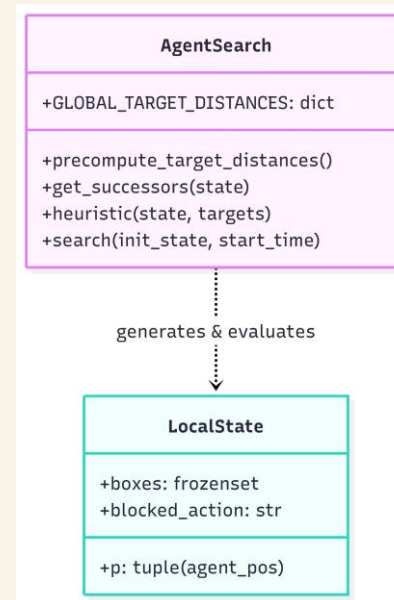
7. COMPETITIVE SEARCH AGENTS:

3. Agent Class Diagram

AgentSearch pre-computes target distances, generates successors, evaluates the matching heuristic, and controls the timed search.

LocalState stores the box configuration, the agent position, and any blocked action used to discourage repeated deadlocks.

Relationship: AgentSearch generates and evaluates LocalState successors.



8. COMPETITIVE MECHANICS & GUI:

1. OOP Model — State and Game Logic

CompetitiveState: AI-safe arena snapshot containing p1, p2, b_free, b1, b2, t, and blocked_action.

CompetitiveGameLogic: Owns map parsing, state transitions, simultaneous collision physics, scoring, and box ownership.

load_map(file_name): Parses %, D, B, 1, and 2 from native layouts.

execute_simultaneous_moves(a1, a2): Resolves swaps, congestion, conflicting pushes, coordinates, and ownership transfer.

get_current_state(agent_id): Returns an isolated CompetitiveState and injects blocked_action when livelock is detected.

reset_logic(step_limit): Clears the arena for a new match.

8. COMPETITIVE MECHANICS & GUI:

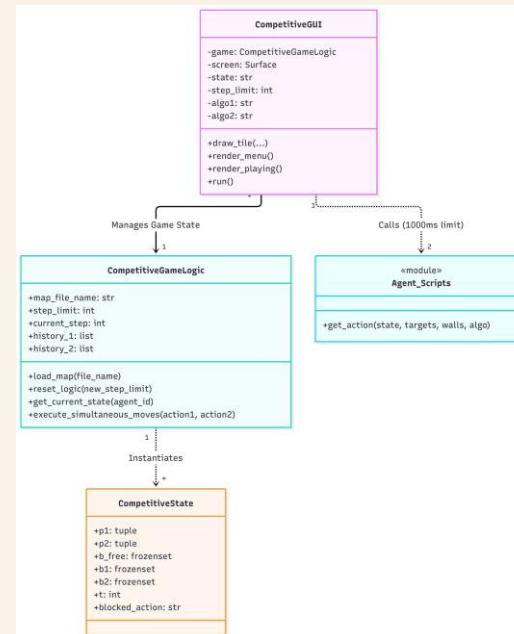
2. CompetitiveGUI

Role: Handles all Pygame rendering and input while keeping agent scripts isolated.

Core attributes: Game-logic reference, tile size, fonts, menu controls, algorithm selectors, step limit, and MENU / PLAYING state.

Renderers: draw_tile(), render_menu(), and render_playing().

run(): Fetches both AI actions with the 1,000 ms rule, handles menu input, SPACE to pause, and ESC to return.



8. COMPETITIVE MECHANICS & GUI:

3. Execute Simultaneous Moves — Pseudocode

```
FUNCTION Execute_Simultaneous_Moves(action_1, action_2):  
  target_1, push_1 = calculate_destination(p1, action_1)  
  target_2, push_2 = calculate_destination(p2, action_2)  
  IF target hits wall: cancel_movement()  
  IF same target OR direct position swap:  
    cancel_both_movements()  
  IF pushing same box OR boxes into same cell:  
    cancel_both_movements()  
  update_agent_coordinates(target_1, target_2)  
  IF box pushed to target:  
    assign_to_agent_color(box)  
  ELSE IF box pushed off target:  
    revert_to_neutral_status(box)
```

8. COMPETITIVE MECHANICS & GUI:

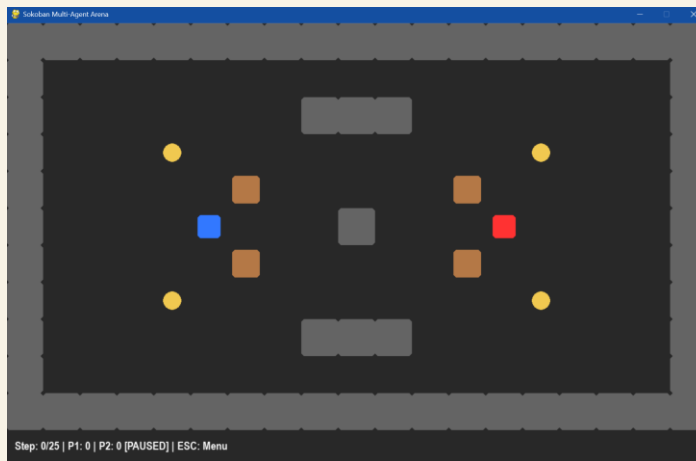
4. Advantages

- Decoupled logic and UI keep AI evaluation separate from rendering.
- Three ownership sets enable $O(1)$ color rendering and score calculation.
- History arrays detect repeated cancellations and help break temporal livelocks.
- MENU → PLAYING supports live algorithm and step-limit selection.

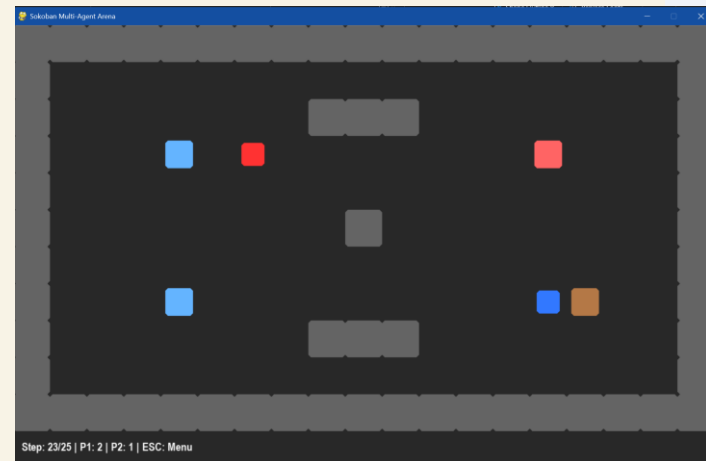
5. Limitations

- Strict simultaneous resolution can waste turns when agents repeatedly cancel each other in narrow corridors.
- Running two search agents inside the GUI loop is CPU intensive.
- Large maps may trigger the 1.0 s fallback and return “Wait” to keep the interface responsive.

8. COMPETITIVE MECHANICS & GUI:



Paused arena — A* vs GBFS



Live match — ownership and score state



This is the end
of our
presentation.
Thank you for
listening.